

Module 4: Intermediate Code & Runtime

Three-Address Code (Quadruples/Triples), Arrays, Backpatching & Activation Records

Module IV: Control Flow, Backpatching & Runtime Environments — 9-Topic Master Guide

Topic 27: Intermediate Code for Boolean Expressions (Short-Circuit Evaluation)

Topic 29: Backpatching Mechanics (`makelist`, `merge`, `backpatch`, `truelist`, `falselist`)

Topic 31: Translation of Procedure Calls & Returns (`param`, `call`, `return`)

Topic 33: Activation Records Anatomy (The 7 P-R-C-A-S-L-T Frame Fields)

Topic 35: Access to Non-Local Variables (Access Links vs. Displays Technique)

Topic 28: Control Flow Translation (`if-then`, `if-then-else`, `while`, `for`, `switch`)

Topic 30: Translation of Forward & Backward Jump Control Structures

Topic 32: Runtime Storage Organization (Code, Static, Heap & Call Stack)

Topic 34: Storage Allocation Strategies (Static, Stack & Heap Allocation)

Topic 27 & 28: Boolean Expressions & Control Flow Translation

In programming languages, **Boolean Expressions** serve two fundamentally distinct operational purposes:

Numerical Value Representation: Computing explicit logical truth values stored into boolean variables (e.g., `flag = (a > b)` where `flag` evaluates to integer `1` or `0`).

Flow-of-Control Branching: Governing the conditional execution trajectory in decision and looping statements (e.g., `if (a > b) ...` or `while (x <= y) ...`).

Short-Circuit Evaluation Semantics

Compilers generate highly optimized short-circuit conditional code that avoids evaluating redundant subexpressions:

- In an **OR expression** (B_1 **or** B_2): If B_1 evaluates to **true**, the overall expression is immediately known to be **true**; evaluation of B_2 is completely bypassed.
- In an **AND expression** (B_1 **and** B_2): If B_1 evaluates to **false**, the overall expression is immediately known to be **false**; evaluation of B_2 is completely bypassed.

Step-by-Step Solved Problem: Generating Three-Address Code for Complex Control Flow

Source Statement: `if (a < b || c < d && e < f) x = y + 1; else x = y - 1;`

```
100: if a < b goto 104      ; Short-circuit OR: if (a < b) is TRUE → jump directly to true-body
101: if c < d goto 102      ; If (c < d) is TRUE → must evaluate (e < f)
102: goto 107              ; If (c < d) is FALSE → short-circuit AND fails → jump to else-body
103: if e < f goto 104      ; If (e < f) is TRUE → jump to true-body
104: goto 107              ; If (e < f) is FALSE → jump to else-body
105: t1 = y + 1            ; True-body statement
106: x = t1
107: goto 110              ; Jump past else-body
108: t2 = y - 1            ; Else-body statement
109: x = t2
110: ...                  ; Next statement
```

28.1 Translation of `switch-case` Statements: 3 Compilation Strategies

Compilation Strategy	Operating Mechanism	Ideal Scenario & Time Complexity
1. Linear Comparison Sequence	Emits sequential `if (E == V1) goto L1; if (E == V2) goto L2; ...`	Small number of case branches (≤ 4 cases); $O(n)$ search time.
2. Jump Table (Direct Indexing)	Creates a contiguous array of code pointers indexed directly by value ($E - V_{\min}$).	Dense, tightly clustered case values ($[1, 2, 3, 4, 5]$); $O(1)$ instant jump time.
3. Binary Search Tree (Hash Table)	Emits a balanced binary search decision tree of conditional branches over sorted case values.	Large, sparse case values ($[10, 500, 10000, 500000]$); $O(\log n)$ search time.

Topic 29 & 30: Backpatching Mechanics in Single-Pass Compilers

In a single-pass compiler, when the code generator emits a conditional or unconditional branch instruction, the **target jump address** is frequently unknown because the target statement has not yet been parsed. **Backpatching** resolves this problem by storing lists of instruction indices requiring branch targets, and filling in (backpatching) the actual statement labels once they become known.

The 3 Fundamental Backpatching Helper Functions: 1. **makelist(i)**: Constructs a newly initialized list containing exactly one element: the integer index i of a newly emitted branch instruction. 2. **merge(p_1, p_2)**: Concatenates two jump index lists p_1 and p_2 , returning a unified combined list pointer. 3. **backpatch(p, L)**: Traverses every instruction index stored in list p , inserting statement label L as the explicit target jump address.

29.1 Syntax-Directed Translation Scheme for Boolean Backpatching:

Grammar Production	Semantic Action Rule for Backpatching
$B \rightarrow B_1 \text{ or } M B_2$	$\text{backpatch}(B_1.\text{falselist}, M.\text{quad})$ $B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist})$ $B.\text{falselist} = B_2.\text{falselist}$
$B \rightarrow B_1 \text{ and } M B_2$	$\text{backpatch}(B_1.\text{truelist}, M.\text{quad})$ $B.\text{truelist} = B_2.\text{truelist}$ $B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist})$
$B \rightarrow \text{not } B_1$	$B.\text{truelist} = B_1.\text{falselist}$ $B.\text{falselist} = B_1.\text{truelist}$
$B \rightarrow (B_1)$	$B.\text{truelist} = B_1.\text{truelist}$ $B.\text{falselist} = B_1.\text{falselist}$
$B \rightarrow \text{id}_1 \text{ relop id}_2$	$B.\text{truelist} = \text{makelist}(\text{nextquad})$ $B.\text{falselist} = \text{makelist}(\text{nextquad} + 1)$ $\text{\texttt{\text{emit}(\text{"if " + \mathbf{id}_1.\text{name} + \mathbf{relop}.\text{op} + \mathbf{id}_2.\text{name} + \text{"goto _")})}}$ $\text{\texttt{\text{emit}(\text{"goto _")}}$
$M \rightarrow \epsilon$	$M.\text{quad} = \text{nextquad}$

Topic 31: Translation of Procedure Calls & Parameter Passing

Procedure calls require coordinating data passing, machine state preservation, stack frame allocation, and execution transfer between the **Caller** and **Callee** routines:

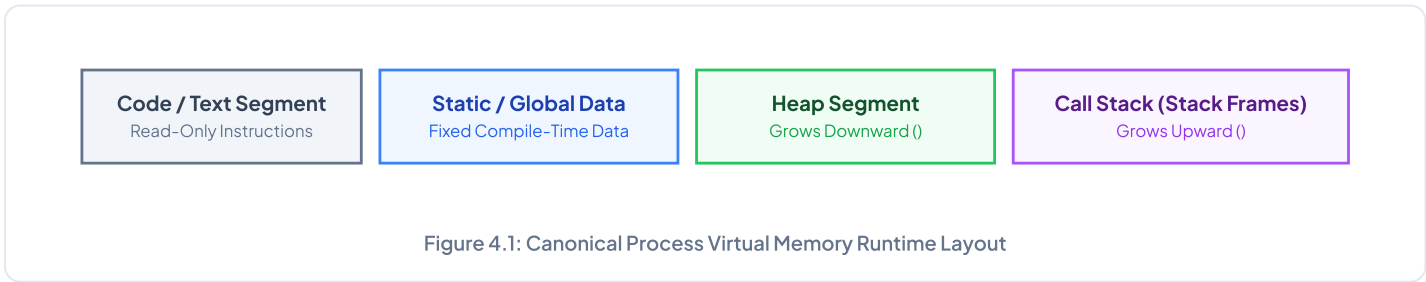
Three-Address Code Sequence for Procedure Call `result = compute_sum(a + 10, b * 2)`

```
t1 = a + 10
param t1      ; Push actual argument 1 onto parameter list
t2 = b * 2
param t2      ; Push actual argument 2 onto parameter list
call compute_sum, 2 ; Transfer control to procedure compute_sum (2 parameters)
result = retval ; Retrieve return value from CPU return register
```

Parameter Passing Method	Operating Mechanism	Key Semantic Characteristics
1. Call-by-Value (C, Java Primitives)	Actual argument expression is evaluated and a local copy of the r-value is passed into the callee's stack frame.	Modifications inside the callee have zero effect on the caller's variable.
2. Call-by-Reference (C++ '&', Fortran)	The l-value (memory address) of the actual argument is passed directly.	Changes made to the formal parameter immediately mutate the caller's actual variable.
3. Call-by-Restore / Copy-In Copy-Out	Actual value is copied in at function entry; final value is copied back out to caller's variable upon function return.	Behaves like call-by-reference except in cases of variable aliasing.
4. Call-by-Name (Algol 60)	The actual argument expression is substituted textually into the callee body as a parameterless function (called a Thunk).	Re-evaluates the argument expression every single time the formal parameter is accessed.

Topic 32 & 33: Runtime Storage Organization & Activation Records

The operating system allocates a continuous block of virtual memory to the executing process, organized into four canonical logical memory segments:



33.1 Exhaustive Anatomy of an Activation Record (Stack Frame):

Activation Record Field	Internal Purpose & Operational Contents	Managing Party
1. Actual Parameters	Holds argument values passed from the caller to the callee.	Caller Routine
2. Returned Value	Reserved memory cell where the callee writes the function return result.	Callee Routine
3. Control Link (Dynamic Link)	Pointer pointing to the base address of the caller's activation record . Used to restore caller's stack frame upon return.	Callee Routine
4. Access Link (Static Link)	Pointer pointing to the activation record of the lexical enclosing parent . Used to resolve non-local variable references.	Callee Routine
5. Saved Machine Status	Preserves CPU Program Counter (PC), CPU registers, stack base pointers, and condition flags prior to procedure invocation.	Callee Routine
6. Local Variables	Stores local scalar variables and fixed-size arrays declared within the procedure body.	Callee Routine
7. Temporaries	Holds intermediate values generated during expression evaluation, array address offsets, and register spills.	Callee Routine

 Memory Hook: The 7 Fields of an Activation Record (P-R-C-A-S-L-T)

Parameters → Returned value → Control link → Access link → Saved status → Local data → Temporaries

Topic 34 & 35: Storage Allocation & Non-Local Data Access (Displays)

Storage Strategy	Operating Mechanism	Advantages & Critical Tradeoffs
1. Static Allocation	All memory addresses for all variables and procedures are permanently fixed at compile time (Fortran 77).	Zero runtime allocation overhead; cannot support recursion or dynamic data structures.
2. Stack Allocation	Stack frames are pushed onto the call stack upon procedure call and popped upon procedure return (LIFO).	Supports unbounded recursion; local variable lifetimes strictly bounded by function lifetime.
3. Heap Allocation	Dynamically allocated in arbitrary order ('malloc'/'new') and reclaimed via explicit 'free' or Garbage Collection.	Enables dynamic data structures (trees, graphs); incurs memory fragmentation and allocation overhead.

Non-Local Variable Access: Static Links vs. Displays Array Technique

- Access Links (Static Links):** Chained pointer traversal. Accessing a non-local variable defined k lexical nesting levels away requires dereferencing k successive pointers ($O(k)$ time complexity).
- Displays Array:** An auxiliary global array of pointers `d[0..max_depth]` where `d[i]` directly points to the most recently active activation record at lexical nesting depth i . Provides guaranteed $O(1)$ instant access to any non-local variable at any nesting depth!



Q1. Explain Backpatching with complete Syntax-Directed Translation Scheme for a `while (B) do S` loop. (10 Marks)

Production: $S \rightarrow \text{while } M_1 \text{ B do } M_2 S_1$

Semantic Action Rules:

$\text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad});$ // Loop body branches back to condition

$\text{backpatch}(B.\text{truelist}, M_2.\text{quad});$ // True condition branches into loop body

$S.\text{nextlist} = B.\text{falselist};$ // False condition exits while loop

$\text{emit}(\text{"goto " } M_1.\text{quad});$ // Emit unconditional loop back-edge

Q2. Draw and explain the complete anatomy of an Activation Record (Stack Frame) with all 7 fields. (10 Marks)

An Activation Record manages the execution state of a procedure instance. The 7 standard fields are:

1. **Actual Parameters:** Arguments passed into the procedure.
2. **Returned Value:** Result returned to caller.
3. **Control Link (Dynamic Link):** Pointer to caller's activation record for stack unwinding.
4. **Access Link (Static Link):** Pointer to enclosing static scope for non-local variables.
5. **Saved Machine Status:** Saved Program Counter, CPU registers, and status flags.
6. **Local Data:** Local variables declared in procedure body.
7. **Temporaries:** Temporary evaluation cells for complex expressions.

Q3. Compare Call-by-Value, Call-by-Reference, and Call-by-Name parameter passing methods with code examples. (8 Marks)

- **Call-by-Value:** R-value copied to local parameter. Changes inside callee do not affect caller.
- **Call-by-Reference:** L-value (address) passed directly. Changes inside callee immediately mutate caller's variable.
- **Call-by-Name:** Argument substituted textually as a parameterless Thunk function. Re-evaluated upon every access.

Q4. How does the Displays array technique achieve $O(1)$ access to non-local variables compared to static links? (8 Marks)

Static links require traversing a linked list of k parent pointers ($O(k)$ time). The **Displays** technique maintains a global array `d[i]` pointing directly to the active frame at lexical depth i . Any non-local variable at depth i is accessed in single-step dereference `d[i] -> offset`, achieving guaranteed $O(1)$ lookup time!

Q5. Explain the Caller and Callee Calling Sequences and register saving conventions in x86-64 / MIPS. (8 Marks)

- **Caller Calling Sequence:** (1) Evaluates actual arguments; (2) Pushes caller-saved registers; (3) Emits `param` instructions; (4) Emits `call` which pushes return address.
- **Callee Prologue:** (1) Saves old frame pointer (`push rbp`); (2) Sets new frame pointer (`mov rbp, rsp`); (3) Allocates stack space for local variables (`sub rsp, framesize`); (4) Saves callee-saved registers.
- **Callee Epilogue:** (1) Writes return value to RAX register; (2) Restores callee-saved registers; (3) Restores stack pointer (`mov rsp, rbp; pop rbp`); (4) Emits `ret`.

Topic 35.2: Comprehensive Worked Control Flow & Runtime Numerical Traces

Solved Problem 1: Complete Three-Address Code for Nested While Loop

Source Statement:

```
while (i < 10 && j > 0) {  
    if (a[i] == b[j]) {  
        x = x + 1;  
    } else {  
        y = y + 1;  
    }  
    i = i + 1;  
    j = j - 1;  
}
```

Generated Three-Address Code:

```
100: if i < 10 goto 102  
101: goto 120          ; Exit loop if (i < 10) is False  
102: if j > 0 goto 104  ; Check second condition  
103: goto 120          ; Exit loop if (j > 0) is False  
104: t1 = i * 4         ; Array offset a[i]  
105: t2 = a[t1]  
106: t3 = j * 4         ; Array offset b[j]  
107: t4 = b[t3]  
108: if t2 == t4 goto 111  
109: goto 114          ; Branch to else-body  
110: goto 114  
111: t5 = x + 1         ; True-body: x = x + 1  
112: x = t5  
113: goto 116  
114: t6 = y + 1         ; Else-body: y = y + 1  
115: y = t6  
116: t7 = i + 1         ; Loop step: i = i + 1  
117: i = t7  
118: t8 = j - 1        ; Loop step: j = j - 1  
119: j = t8  
120: goto 100          ; Loop back-edge to condition  
121: ...              ; Next statement
```

Solved Problem 2: Displays Array Non-Local Variable Access Execution Trace

Consider lexical nesting levels: `Main` (Depth 1) $\rightarrow$ `Procedure P` (Depth 2) $\rightarrow$ `Procedure Q` (Depth 3).

Displays Array Structure:

- `d[1]` points to active Activation Record of `Main`
- `d[2]` points to active Activation Record of `Procedure P`
- `d[3]` points to active Activation Record of `Procedure Q`

When `Procedure Q` references variable `x` declared in `Main` (at offset 16 from `Main`'s FP):

$$\text{Memory Target Address} = d[1] + 16 \quad (\text{Direct } O(1) \text{ Instant Lookup!})$$

When `Procedure Q` calls `Procedure R` at depth 2, the old `d[2]` pointer is saved in `R`'s frame and `d[2]` is updated to point to `R`. Upon return, `d[2]` is restored to `P`.

Q6. Compare Static, Stack, and Heap Memory Allocation strategies across 5 dimensions. (10 Marks)

1. **Allocation Timing:** Static is fixed at compile-time; Stack occurs upon function entry (LIFO); Heap occurs on dynamic programmer request (`malloc`).
2. **Recursion Support:** Static cannot support recursion; Stack handles unbounded recursion seamlessly; Heap supports dynamic object graph structures.
3. **Data Lifetime:** Static persists for entire program run; Stack is strictly bounded by function return; Heap persists until explicit `free` or Garbage Collection.
4. **Overhead:** Static has zero runtime overhead; Stack has minimal pointer adjustment (`sub rsp, framesize`); Heap incurs memory fragmentation and allocator search costs.
5. **Deallocation:** Static is reclaimed on program termination; Stack automatically pops on return; Heap requires explicit deallocation or Garbage Collection.

Q7. Explain the Control Link (Dynamic Link) vs. Access Link (Static Link) in Activation Records. (8 Marks)

- **Control Link (Dynamic Link):** Points to the base address of the **caller's activation record**. It reflects the dynamic call history and is used to restore the caller's stack frame pointer upon function return.
- **Access Link (Static Link):** Points to the activation record of the **lexical enclosing parent procedure** in the source code. It reflects static lexical nesting and is used to resolve non-local variable references at runtime.

Q8. Explain Short-Circuit Boolean Evaluation with code generation templates for `if (B) S1 else S2`. (8 Marks)

Short-circuit evaluation stops evaluating boolean expressions as soon as the final truth value is determined.

Template for `if (B) S1 else S2`:

1. Translate boolean condition B , generating `B.truelist` and `B.falselist`.
2. `backpatch( $B$ .truelist, label( $S_1$ )).`
3. `backpatch( $B$ .falselist, label( $S_2$ )).`
4. Emit unconditional `goto` at the end of S_1 to jump past S_2 .

Q9. How are `switch-case` statements compiled into Three-Address Code? Compare Jump Tables vs. Binary Search Trees. (8 Marks)

- **Linear Comparison Sequence:** Emits sequential `if (E == V) goto L` for ≤ 4 cases ($O(n)$ time).
- **Jump Table (Direct Array Indexing):** Creates an array of branch targets indexed by $(E - V_{\min})$ for dense integer ranges ($O(1)$ time).
- **Binary Search Decision Tree:** Emits a balanced binary search comparison tree for large, sparse case values ($O(\log n)$ time).

Q10. Explain the Garbage Collection Mark-and-Sweep algorithm for Heap Memory Management. (8 Marks)

The **Mark-and-Sweep** algorithm reclaims unreachable dynamic heap memory in 2 phases:

1. **Mark Phase:** Starts from Root Set (global variables, stack pointers, CPU registers) and performs a Graph Traversal (DFS/BFS), setting a `marked = true` bit on all reachable heap objects.
2. **Sweep Phase:** Linearly scans the entire heap space. Unmarked blocks are added to the Free List for future allocation; marked blocks have their mark bits cleared for the next cycle!

Topic 35.3: Advanced Runtime Memory Management & Parameter Passing Traces

In modern runtime environments, memory allocation and procedure execution depend heavily on the target hardware architecture, CPU calling conventions (such as System V AMD64 ABI), and memory management strategies.

Memory Arena	Hardware Characteristics	Growth Direction	Management Protocol
Code / Text	Read-only, executable pages mapped directly from executable binary.	Static fixed bounds	Operating System kernel memory pager.
Static / Data	Read-write, global and static variables initialized at compile time.	Static fixed bounds	Operating System loader initialization.
Heap Segment	Dynamic memory allocated via system calls (`sbrk`, `mmap`).	Grows Upward (↑)	`malloc`/`free` allocator with Buddy System.
Call Stack	LIFO stack frames allocated and freed on function entry and exit.	Grows Downward (↓)	Hardware Stack Pointer (`rsp`) & Frame Pointer (`rbp`).

Step-by-Step Solved Problem: Complete Calling Sequence Machine Code Flow

Consider a function call `result = calculate(arg1, arg2)` on x86-64 Architecture:

```
; 1. CALLER SEQUENCE:
MOV RDI, [rbp - 8]      ; Place actual argument 1 into 1st argument register RDI
MOV RSI, [rbp - 16]     ; Place actual argument 2 into 2nd argument register RSI
CALL calculate          ; Pushes return address (RIP) to stack, jumps to calculate

; 2. CALLEE PROLOGUE:
calculate:
PUSH RBP                ; Save dynamic control link (caller's base pointer)
MOV RBP, RSP            ; Establish new stack frame base pointer
SUB RSP, 32             ; Allocate 32 bytes on stack for local variables and temporaries

; 3. CALLEE BODY & EPILOGUE:
; ... execute procedure instructions ...
MOV RAX, [rbp - 4]      ; Write return value into RAX register
MOV RSP, RBP           ; Deallocate local frame variables
POP RBP                ; Restore caller's base pointer (control link)
RET                    ; Pops return address from stack into RIP register

; 4. CALLER RESUMPTION:
MOV [rbp - 24], RAX     ; Store returned value from RAX into variable result
```


Topic 35.4: Comprehensive Calling Conventions & Dynamic Heap Allocators

A **Calling Convention** defines the low-level binary contract between caller and callee subroutines. It specifies how parameters are passed (registers vs. stack), how return values are delivered, which CPU registers must be preserved across calls (callee-saved vs. caller-saved), and who cleans up the parameter stack space.

Calling Convention	Parameter Passing Mechanism	Stack Cleanup Responsibility	Representative Platforms
cdecl (Standard C)	All parameters pushed onto stack right-to-left.	Caller cleans stack (`add rsp, N`).	x86 32-bit Linux / Windows C compilers.
stdcall	All parameters pushed onto stack right-to-left.	Callee cleans stack (`ret N`).	Windows Win32 API functions.
fastcall	First 2 arguments passed in `ECX`, `EDX`; remainder on stack.	Callee cleans stack.	Performance-critical x86 libraries.
System V AMD64 ABI	First 6 integer/pointer args in `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`; floats in `XMM0..7`.	Caller cleans stack frame.	x86-64 Linux, macOS, FreeBSD, GCC, Clang.

Step-by-Step Solved Problem: Buddy System Dynamic Memory Allocation & Merging

Suppose a runtime heap starts with a single continuous block of size **1024 KB** (addressed from 0 to 1023). Trace allocations and deallocations:

- Request A (Alloc 70 KB):** Smallest power of 2 is 128 KB. The 1024 KB block splits into two 512 KB buddies → split to 256 KB → split to two 128 KB blocks. Block [0..127] is allocated to *A*.
- Request B (Alloc 200 KB):** Smallest power of 2 is 256 KB. Block [256..511] is allocated to *B*.
- Request C (Alloc 110 KB):** Smallest power of 2 is 128 KB. Remaining buddy block [128..255] is allocated to *C*.
- Free A (Releases [0..127]):** Block [0..127] is marked free. Its buddy [128..255] is currently allocated to *C*, so they cannot merge yet.
- Free C (Releases [128..255]):** Both buddies [0..127] and [128..255] are now free! They coalesce instantly into a unified 256 KB block [0..255]. This new block's buddy [256..511] is occupied by *B*, so coalescing halts.

Solved Problem: Backpatching Execution Trace for `for` Loop with `break`

Source Statement: ``for (i = 0; i < 100; i++) { if (a[i] == 0) break; sum += a[i]; }``

```
100: i = 0                ; Initialization
101: if i < 100 goto 103    ; Loop condition
102: goto 112              ; falselist: exits for loop
103: t1 = i * 4            ; Array address offset a[i]
104: t2 = a[t1]
105: if t2 == 0 goto 112    ; break statement: jumps directly to exit label 112
106: goto 107
107: t3 = i * 4
108: t4 = a[t3]
109: sum = sum + t4        ; Loop body
110: i = i + 1             ; Step increment
111: goto 101              ; Back-edge to condition check
112: ...                   ; Target of backpatch(falselist + break_list, 112)
```

Topic 35.5: Object-Oriented Runtime Layout & Virtual Method Tables (Vtables)

In object-oriented languages (such as C++, Java, and C#), member methods may be dynamically overridden in derived subclasses (polymorphism). The compiler realizes **Dynamic Dispatch** via an auxiliary data structure called a **Virtual Method Table (Vtable)**.

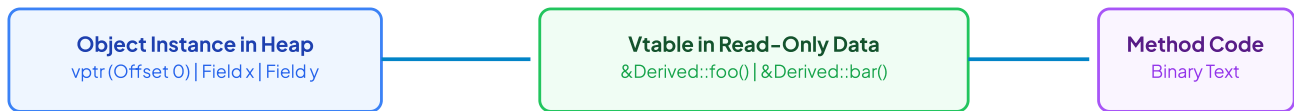


Figure 4.2: Object Instance Memory Layout with Virtual Method Table Pointer ('vptr')



Step-by-Step Solved Problem: Dynamic Dispatch Assembly Code Generation

Consider a polymorphic method call `obj->draw()` where `draw()` is at virtual table index 1 (8 bytes offset):

```
MOV RDI, [rbp - 8]    ; Load obj pointer into 1st argument register RDI (the 'this' pointer)
MOV RAX, [RDI]        ; Dereference obj to fetch its Virtual Table Pointer (vptr)
MOV RAX, [RAX + 8]    ; Fetch address of draw() from Vtable slot 1
CALL RAX              ; Indirect procedure call to the concrete method implementation!
```

Complexity: Dynamic dispatch executes in guaranteed $O(1)$ time with only 2 memory dereferences!